TCSS 321 – Discrete Structures

Algorithms and Loop Invariants

Sets and Sequences

Definition: A sequence is an ordered set, $(a_1, a_2, \dots, a_n)$, such that $a_i \in S$ for some set S.

- A sequence differs from a set in two important ways:
  1. Order matters in a sequence.
  2. Duplicate elements are permitted.
- We use round braces $()$ for sequences.
- A sequence is also commonly called a tuple in math.
  - Double, triple, quadruple, quintuple, etc.
  - In general a n-tuple is a sequence with n elements.

Examples: Here are some short sequences of integers.

$$(1,2,3,4,5) \neq (5,4,3,2,1)$$
$$(1,1,2,3,5,8) \neq (1,2,3,5,8)$$
$$(1,2,4,8,16,32)$$

Alphabets and Strings

Definition: A set of characters (or symbols) is called an alphabet.

- We typically use capital Greek characters like Σ and Γ for alphabets.

Examples: Here are some alphabets.

$$\Sigma = \{a, b, c, d, \dots, z\}$$
$$\Gamma = \{\alpha, \beta, \gamma, \delta, \dots, \omega\}$$
$$\Sigma_0 = \{0,1\}$$
$$\Sigma_D = \{A, T, G, C\}$$

Definition: A string is a sequence of characters from an alphabet.

- When stating a string it is common to use “” instead of () and omit the commas.

$$"string"$$
$$"GATTACA"$$

Definition: The empty string, ϵ, is the string with no characters.

Problems and Algorithms

- Consider trying to find the minimum integer in a sequence of integers.

Minimum
Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.
Output: $a_i \in A$ such that $a_i \leq a_j$ for all $a_j \in A$.

Definition: A problem (or information processing problem) is a relationship between the input to the problem and the correct output to the problem.

- The problem does not state how to reach the output from the input.

Definition: An algorithm is a finite sequence of information processing steps that transforms an input to a problem into the correct output for that problem.

- We say an algorithm solves the problem if it is correct for the problem.

Minimum

Minimum
Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.
Output: $a_i \in A$ such that $a_i \leq a_j$ for all $a_j \in A$.

- Can we state a correct algorithm that will solve the minimum problem?

```
Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.
        For i = 2 to n:
                If a_i < m then let m = a_i.
        Endfor
Return m.
```

Pseudocode

```
Minimum (A = (a_1, a_2, …, a_n))
        Let m = a_1.
        For i = 2 to n:
                If a_i < m then let m = a_i.
        Endfor
Return m.
```

- This algorithm is stated in pseudocode.

Definition: Pseudocode is a mix of natural language and code like structures.

- Pseudocode is used for the statement of algorithms whereas machine code is used for the statement of programs in a specific language.
  - The intended audience of pseudocode is other humans who might want to implement an algorithm.
  - The intended audience of code is the computer which is why we must write it in the computer's language.

Algorithm Analysis

```
Minimum (A = (a_1, a_2, ..., a_n))
    Let m = a_1.
    For i = 2 to n:
        If a_i < m then let m = a_i.
    Endfor
Return m.
```

- There are two important questions to ask about any algorithm.
  - Is this algorithm correct?
  - How many operations does this algorithm take on an input of size n? (We'll answer this question later.)

Algorithm Correctness

Definition: An algorithm A is correct for a problem P if for every input of P the algorithm A returns the correct output for P.

- So an algorithm for the Minimum problem is only correct if the algorithm always returns the minimum of the input list.

- In order to establish correctness of an algorithm we must prove it correct.
  - Such a proof is called a proof of correctness.
  - To do this we need to show that for any input the algorithm returns the correct output.

Correctness

```
Minimum
Input: A sequence of integers A = (a_1, a_2, ..., a_n).
Output: a_i ∈ A such that a_i ≤ a_j for all a_j ∈ A.

Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.
        For i = 2 to n:
                If a_i < m then let m = a_i.
        Endfor
Return m.
```

- To prove our algorithm correct we must show that for any input $A = (a_1, a_2, \dots, a_n)$ we return $m = a_i \in A$ such that $m \leq a_j$ for all $a_j \in A$.
  - Generality: We will assume an arbitrary input so that the proof will apply to any possible input.

Proof of Correctness

```
Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.
        For i = 2 to n:
                If a_i < m then let m = a_i.
        Endfor
Return m.
```

Theorem: If Minimum is called on input $A = (a_1, a_2, \ldots, a_n)$, a set of integers, then Minimum returns $m = a_i$ such that $m \leq a_j$ for all $a_j \in A$.

- Because this proof must apply for any $n \in \mathbb{N}$ we potentially have to prove infinite cases.
- The proof strategy of induction allows you to prove infinite cases with a single proof.
- Inductive proofs have a three primary steps: the base case, the inductive hypothesis and the inductive case.

Loop Invariant

- In order to prove the correctness of an algorithm using a loop we prove what is called the loop invariant.
  - Simply, invariant means "unchanging". In computer science (and math) it means a property that remains unchanged after some transformation is applied.

Definition: A loop invariant is a property that remains unchanged after an iteration of the loop.

```
Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.
        For i = 2 to n:
                If a_i < m then let m = a_i.
        Endfor
Return m.
```

- What property of m remains unchanged after iteration i of the loop? (Hint: It should help us show that after the last iteration of the loop m is the minimum of A.)

Loop Invariant: Minimum

Theorem: After iteration i of the loop $m \in A$ and $m \leq a_j$ for all $a_j \in (a_1, a_2, \ldots, a_i)$.

- We will prove this in two parts.
  - First, we will prove that if $i = 1$ then the theorem is true. (This is called the base case).
  - Secondly, we will show that if the theorem holds for iteration i then it also holds for iteration $i + 1$. (This is called the inductive case.)
- Notice that if we prove both propositions then we have proven the theorem for any i (in particular n).
  - We establish a base of $i = 1$.
  - Then since if it is true for i it is true for $i + 1$ we immediately establish $i = 2$.
  - $i = 2$ implies $i = 3$, which implies $i = 4$, which … implies $i = n$.
- This two-stage proof is called mathematical induction.

Corollary: The Minimum algorithm is correct for the Minimum problem.

Loop Invariant: Minimum

Theorem: After iteration i of the `for` loop in Minimum $m \in A$ and $m \leq a_j$ for all $a_j \in (a_1, \ldots, a_i)$.

Proof: (by induction on i)

Base case: ($i = 1$, i.e. before the loop begins)

Prior to the loop $m = a_1 \in A$. Since there is only a single element in (a_1) and $m \leq a_1$ it is the case that $m \leq a_j$ for all $a_j \in (a_1)$.

Inductive Hypothesis: Assume prior to iteration $i + 1$ that $m \in A$ and $m \leq a_j$for $a_j \in (a_1, \ldots, a_i)$.

Inductive Step: $(i \geq 1)$

We need to show that after iteration $i + 1$ it is the case that $m \in A$ and $m \leq a_j$ for $a_j \in (a_1, \ldots, a_{i+1})$. By the inductive hypothesis we already know that prior to iteration $i + 1$ it is the case that $m \in A$ and $m \leq a_j$ for $a_j \in (a_1, \ldots, a_i)$. So we must show that after iteration $i + 1$ it is the case that $m \in A$ and $m \leq a_{i+1}$.

Case 1: $m \leq a_{i+1}$

In this case we do not change m and so this case is trivially true.

Case 2: $m > a_{i+1}$

In this case we set $m = a_{i+1}$ so it is clear that $m \in A$ and $m \leq a_{i+1}$. Note that since $a_{i+1} < m \leq a_j$ for all $a_j \in (a_1, \ldots, a_i)$ after assigning $m = a_{i+1}$ this is still true.

By induction on i we have shown that after iteration i of the `for` loop in Minimum $m \in A$ and $m \leq a_j$ for all $a_j \in (a_1, \ldots, a_i)$. ■

Proof of Correctness: Minimum

Minimum
Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.
Output: $a_i \in A$ such that $a_i \leq a_j$ for all $a_j \in A$.

Corollary: The Minimum algorithm is correct for the Minimum problem.

Proof: Assume our input is an arbitrary sequence of integers $A = (a_1, a_2, \ldots, a_n)$.

According to the loop invariant after iteration i of the loop $m \in A$ and $m \leq a_j$ for $a_j \in (a_1, a_2, \ldots, a_i)$.

In particular, after iteration n of the loop $m \in A$ and $m \leq a_j$ for $a_j \in (a_1, a_2, \ldots, a_n)$.

After iteration n of the loop we return m so we return $m \in A$ and $m \leq a_j$ for $a_j \in (a_1, a_2, \ldots, a_n)$. ■

Sequential Search

Search

Input: A sequence of integers $A = (a_1, a_2, \cdots, a_n)$ and a target $t \in \mathbb{Z}$.

Output: i, if $a_i = t$

-1, if no i exists such that $a_i = t$

- Can we:
  1. Develop a correct algorithm for this problem?
  2. Prove the algorithm correct?
  3. Count the number of operations carried out by the algorithm? (Coming soon!)

Sequential Search

Search
Input: A sequence of integers $A = (a_1, a_2, \cdots, a_n)$ and a target $t \in \mathbb{Z}$.
Output: i, if $a_i = t$
-1, if no i exists such that $a_i = t$

- Can we develop an algorithm for this problem?

```
Search (A = (a_1, a_2, ..., a_n), t)
    For i = 1 to n:
        If a_i = t then return i.
    Endfor
Return −1.
```

Loop Invariant: Sequential Search

```
Search (A = (a_1, a_2, ⋯, a_n), t)
      For i = 1 to n:
            If a_i = t then return i.
      Endfor
Return −1.
```

- What is the loop invariant for this loop?

Theorem: After iteration i of the loop $t \notin (a_1, a_2, \dots, a_i)$ or we have returned $1 \leq j \leq i$ where $a_j = t$.

- Can we prove this by induction?

Loop Invariant: Sequential Search

Theorem: After iteration i of the loop $t \notin (a_1, a_2, \ldots, a_i)$ or we have returned $1 \leq j \leq i$ where $a_j = t$.

Proof: (by induction on i)

Base case: ($i = 0$)

This case is vacuously true. We have not returned yet and so must prove that $t \notin ()$.

Inductive Hypothesis: Assume prior to iteration $i + 1$ of the loop $t \notin (a_1, a_2, \ldots, a_i)$ or we have returned $1 \leq j \leq i$ where $a_j = t$.

Inductive Step: ($i \geq 0$)

By hypothesis we have either returned $1 \leq j \leq i$ where $a_j = t$ or $t \notin (a_1, a_2, \ldots, a_i)$. If we have already returned $1 \leq j \leq i$ where $a_j = t$ we are done. So assume that $t \notin (a_1, a_2, \ldots, a_i)$.

Case 1: $a_{i+1} = t$

We return $i + 1$ where $a_{i+1} = t$ so we are done.

Case 2: $a_{i+1} \neq t$

By hypothesis $t \notin (a_1, a_2, \ldots, a_i)$ and $a_{i+1} \neq t$ so after iteration $i + 1$ it is the case that $t \notin (a_1, a_2, \ldots, a_{i+1})$.

By induction we have shown that after iteration i of the loop $t \notin (a_1, a_2, \ldots, a_i)$ or we have returned $1 \leq j \leq i$ where $a_j = t$. ■

Proof of Correctness: Sequential Search

Search

Input: A sequence of integers $A = (a_1, a_2, \cdots, a_n)$ and a target $t \in \mathbb{Z}$.

Output: i, if $a_i = t$

-1, if no i exists such that $a_i = t$

Corollary: The Sequential Search algorithm is correct for the Search problem.

Proof: Assume the input is an arbitrary list of integers $A = (a_1, a_2, \dots, a_n)$ and an arbitrary target integer $t \in \mathbb{Z}$.

Case 1: Assume there is no $a_i = t$. Then by the loop invariant after iteration i either $t \notin (a_1, a_2, \dots, a_i)$ or we have returned $a_i = t$. Since there is no $a_i = t$ we know that $t \notin (a_1, a_2, \dots, a_i)$.

In particular, after iteration n we know that $t \notin (a_1, a_2, \dots, a_n)$. At this point we return -1.

Case 2: Assume there is an $a_i = t$ and assume a_i is the first $a_i = t$. By the loop invariant in loop i we will return $a_i = t$.

In both cases the algorithm returns the correct output. ■

String Equals

String Equals
Input: Two strings $A = a_1 a_2 \cdots a_n$ and $B = b_1 b_2 \cdots b_m$.
Output: true, if $A = B$
false, otherwise

- Can we:
  1. Develop a correct algorithm for this problem?
  2. Prove the algorithm correct?
  3. Count the number of operations carried out by the algorithm? (Stay tuned!)

String Equals

String Equals
Input: Two strings $A = a_1 a_2 \cdots a_n$ and $B = b_1 b_2 \cdots b_m$.
Output: true, if $A = B$
false, otherwise

- Can we develop an algorithm for this problem?

```
Equals (A = a_1 a_2 ... a_n, B = b_1 b_2 ... b_m)
        If n ≠ m return false.
        For i = 1 to n:
                If a_i ≠ b_i then return false.
        Endfor
Return true.
```

Loop Invariant: String Equals

```
Equals (A = a_1 a_2 ⋯ a_n, B = b_1 b_2 ⋯ b_m)
      If n ≠ m return false.
      For i = 1 to n:
            If a_i ≠ b_i then return false.
      Endfor
Return true.
```

- What is the loop invariant for this loop?

Theorem: After iteration i of the loop $a_j = b_j$ for all $1 \leq j \leq i$ or we have returned false.

- Can we prove this by induction?

Loop Invariant: String Equals

Theorem: After iteration i of the loop $a_j = b_j$ for all $1 \leq j \leq i$ or we have returned false.

Proof: (by induction on i)

Base case: ($i = 0$)

This case is vacuously true. We have either returned false prior to the loop or we must prove that $\epsilon = \epsilon$. (Recall ϵ is the empty string.)

Inductive Hypothesis: Assume prior to iteration $i + 1$ of the loop $a_j = b_j$ for all $1 \leq j \leq i$ or we have returned false.

Inductive Step: ($i \geq 0$)

By hypothesis we have either returned false or $a_j = b_j$ for all $1 \leq j \leq i$. If we have already returned false then we are done. So assume $a_j = b_j$ for all $1 \leq j \leq i$.

Case 1: $a_{i+1} \neq b_{i+1}$

In this case we return false so we are done.

Case 1: $a_{i+1} = b_{i+1}$

By hypothesis $a_j = b_j$ for all $1 \leq j \leq i$ and $a_{i+1} = b_{i+1}$ so $a_j = b_j$ for all $1 \leq j \leq i + 1$.

By induction we have shown that after iteration i of the loop $a_j = b_j$ for all $1 \leq j \leq i$ or we have returned false. ■

Proof of Correctness: String Equals

String Equals
Input: Two strings $A = a_1 a_2 \cdots a_n$ and $B = b_1 b_2 \cdots b_m$.
Output: true, if $A = B$
false, otherwise

Corollary: The Equals algorithm is correct for the String Equals problem.

Proof: Assume our input consists of two arbitrary strings $A = a_1 a_2 \cdots a_n$ and $B = b_1 b_2 \cdots b_m$. Assume $n = m$ (otherwise we correctly output false prior to the loop).

Case 1: Assume $A = B$. Then by the loop invariant after iteration i of the loop $a_j = b_j$ for all $1 \leq j \leq i$ or we have returned false. Since there are no mismatches to return false we know that $a_j = b_j$ for all $1 \leq j \leq i$.

In particular, after iteration n of the loop we know that $a_j = b_j$ for all $1 \leq j \leq n$. At this point we return true.

Case 2: Assume $A \neq B$. Assume $a_i \neq b_i$ is the first pair of characters that do not match. Then by the loop invariant we will return false in iteration i.

In both cases we return the correct output. ■